

[Skip to content](#)

[SETAPESU26](#)

[28_defender](#)

[Repository navigation](#)

[Code](#)

[Issues](#)

[Pull requests](#)

[Actions](#)

[Projects](#)

[Security and quality](#)

[Insights](#)

[SETAPESU26](#)

[28_defender](#)

[Public](#)

[Go to file](#)

[t](#)

[T](#)

[vishwambharaRH](#)

[vishwambharaRH](#)

[edit readme for submission reqs](#)

[031f1ee](#)

[.](#)

[2 weeks ago](#)

[Name](#)

[.gitignore](#)

[push defender](#)

[2 weeks ago](#)

[README.md](#)

[edit readme for submission reqs](#)

[2 weeks ago](#)

[game.py](#)

[push defender](#)

[2 weeks ago](#)

[Repository files navigation](#)

[README](#)

[Defender Repair Lab](#)

This project is a single-file Defender-lite clone using Pygame. It introduces students to camera wrapping, world-space vs. screen-space coordinates, and multi-stage enemy AI using a small,

readable object-oriented codebase.

What's Provided

A working Defender-lite game with:

A player ship that thrusts left/right and adjusts altitude, wrapping around a world several screens wide

Humanoids on the ground that landers try to abduct, carry upward, and turn into a faster "mutant" enemy if they reach the top

A radar strip along the top meant to show where everything is across the whole world

Waves, lives, scoring, and bonus points for rescuing a humanoid mid-fall

It has one deliberate bug and three optional features left as empty functions. You are expected to analyze, interact with an AI assistant, and complete/fix the game to make it fully functional and more interesting.

Use an LLM (e.g. ChatGPT or Claude) as your debugging and pair-programming partner for this lab.

Getting Started

Setup

Make sure you have Python 3.10+ installed.

Install dependencies:

```
pip install pygame
```

Run the game:

```
python game.py
```

Controls: Left/Right to thrust, Up/Down to change altitude, Space to fire, R to reset.

Tasks to Complete

Each task must be completed using an iterative process involving LLM suggestions and your critical code review.

Task 1: Fix the radar bug

The radar strip is supposed to show every humanoid, lander, and the player at their true position across the entire wrapping world, so threats anywhere in the world are visible at a glance. In the current build, the radar only ever shows things clustered near the

player's own position, because it plots each blip's position relative to the player (screen space) instead of its absolute position in the world. Fix `draw_radar` so each blip's horizontal position on the radar reflects its actual place in the world, from one edge to the other, regardless of where the player currently is.

Task 2: Implement `sky_color(wave)`

Called once per frame in `draw`, as `screen.fill(sky_color(self.wave) or (5, 5, 20))`. It receives the current wave number and should return an (r, g, b) color, or `None` to keep the default near-black sky. Idea: brighten or shift hue as waves progress.

Task 3: Implement `on_humanoid_rescued(humanoid)`

Called from `update()` the moment the player catches a falling humanoid (one a lander dropped or released mid-air), right after the 500-point bonus is added and the humanoid is placed safely back on the ground. It receives the rescued Humanoid. Its return value is ignored. Idea: a "+500" popup, or a short invulnerability boost as a reward.

Task 4: Implement `bonus_life_threshold()`

Called every frame in `update()`. It takes no arguments and should return an integer score value, or `None` to disable bonus lives entirely. Whenever the score crosses a multiple of that value for the first time, one life is awarded automatically — the bookkeeping (`self.bonus_awarded`) is already implemented, so you only need to choose the threshold. Idea: return 10000.

Expected Behavior

The player and every enemy wrap smoothly across both edges of the world

The radar shows the whole world at once; only the main viewport centers on the player, not the radar

A lander that finishes carrying a humanoid off the top of the screen becomes a faster, more aggressive mutant

Catching a humanoid mid-fall returns it safely to the ground and awards points; letting it fall too far kills it

The game ends when lives reach zero

Folder Structure

defender/

└─ game.py
└─ README.md

Submission Checklist

Submission is only the following three things:

- A 10-second video of gameplay before your changes, showing the bug/broken behavior
- A 10-second video of gameplay after your changes, showing the bug fixed and the new features working
- The Chat/LLM used page link, with the complete chat history

About

Implementation of Defender in Pygame.

Resources

Readme

Activity

Custom properties

Stars

0 stars

Watchers

0 watching

Forks

15 forks

Report repository

Releases

No releases published

Packages

No packages published

Contributors

1

(1)

@vishwambharaRH

vishwambharaRHVishwambhara R Hebbalalu

Languages

Python

100%

Footer

© 2026 GitHub, Inc.

Footer navigation

Terms

[Privacy](#)
[Security](#)
[Status](#)
[Community](#)
[Docs](#)
[Contact](#)
[Manage cookies](#)
[Do not share my personal information](#)

Yep — this is the **Defender Repair Lab** assignment. You need to modify `game.py` and then submit exactly **three things**:

1. 10-second video **before** changes, showing the radar bug.
2. 10-second video **after** changes, showing the fixed radar + new features.
3. Link to this ChatGPT conversation containing the complete LLM interaction.

The four code tasks are already clearly specified in the README:

- **Task 1:** Fix `draw_radar()`
- **Task 2:** Implement `sky_color(wave)`
- **Task 3:** Implement `on_humanoid_rescued(humanoid)`
- **Task 4:** Implement `bonus_life_threshold()`

Let's do it step-by-step

Since you usually want the exact commands and actions to perform, **don't change anything yet**.

First, open the repository in VS Code and run:

</> Bash

```
git clone <your-repository-url>
cd 28_defender
code .
```

If you've **already cloned the repository**, just open that folder in VS Code.

Then run:

</> Bash

```
python --version
```

and:

```
</> Bash
```

```
pip install pygame
```

Then:

```
</> Bash
```

```
python game.py
```

Important: Before fixing anything, we need to capture the **10-second "before" video**, because the README explicitly requires gameplay before your changes.

Once you've opened the game and confirmed it runs, tell me **"game running"**. Then I'll walk you through the original behavior/video and the code changes **one task at a time**, without redesigning the game.

when i clicked code . there is already a game.py also i cloned it then forked it is that okay?

Yes, that's okay. And seeing `game.py` after running `code .` is **exactly what we expect**.

One small GitHub clarification:

- **Fork** = creates your own copy of the instructor's repository on your GitHub account.
- **Clone** = downloads a repository to your computer.
- Ideally, for this assignment, you should **fork first, then clone your fork**.
- If you cloned first and forked afterward, that's still fixable — we just need to make sure your local repository points to **your fork**, not the original repository.

First, let's check that.

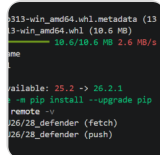
In the VS Code terminal, run:

```
</> Bash
```

```
git remote -v
```

Send me the output.

Don't make any code changes yet. We'll verify that your Git setup is pointing to your fork, then we'll run the original game and record the required **before** video.



Perfect. That output tells us your local repository is currently connected to:

</> Plain text

```
https://github.com/SETAPESU26/28_defender
```

So your **origin** is the original repository, not your fork.

That's not a problem. Since you've already forked it, let's point your local repository to **your fork** before you make any changes. This will make pushing your work straightforward.

Step 1 — Find your fork

Go to your GitHub profile → open your fork of `28_defender` .

Your URL should look like:

</> Plain text

```
https://github.com/YOUR-USERNAME/28_defender
```

Copy that URL.

Step 2 — Change **origin**

In the VS Code terminal, run:

</> Bash

```
git remote set-url origin https://github.com/YOUR-USERNAME/28_defender.git
```

Replace `YOUR-USERNAME` with your actual GitHub username.

Then verify:

</> Bash

```
git remote -v
```

You should now see your username in both the `fetch` and `push` URLs.

Send me the new `git remote -v` output before doing anything else.

yea done

Perfect. 👍 Your Git setup is now pointing to your fork.

Step 3 — Run the original game

In the VS Code terminal, run:

```
</> Bash
```

```
python game.py
```

The Defender game window should open.

Do not edit `game.py` yet. This is the original/broken version, and we need it for the required **10-second before video**.

For the video, show:

- The game running.
- Move the player left/right for a few seconds.
- Fire a few shots if you want.
- Make sure the **radar at the top is visible** so the original radar behavior is captured.

You only need about **10 seconds**.

After you've recorded it, close the game window and tell me "**before video done**". Then we'll inspect `game.py` and start Task 1.

how do iscreen recodrd

Since you're on **Windows**, the easiest way is the built-in Xbox Game Bar.

Record the "before" video

1. Start the game:

```
</> Bash
```

```
python game.py
```

2. When the game window appears, press:

Win + G

3. A Game Bar will appear. Find the **Capture** panel.

4. Click the **Record** button.

Or simply press:

Win + Alt + R

5. Play the game for about **10 seconds**.

- Move left/right.
- Fire a few shots.
- Keep the **radar visible**.

6. Press:

Win + Alt + R

again to stop recording.

Where is the video?

Usually:

File Explorer → Videos → Captures

The file should be an `.mp4`.

You don't need to edit it. **10 seconds is enough**.

Once you've got it, tell me **"before video done"**, and we'll move on to fixing `draw_radar()`.

done recorded

Perfect.  The **before-change video is done**.

Now let's start **Task 1: Fix the radar bug**.

Step 1 — Open `game.py`

In VS Code:

1. Click `game.py` in the Explorer.
2. Press **Ctrl + F**.
3. Search for:

 **Plain text**

```
draw_radar
```

You should find something like:

 **Python**

```
def draw_radar(self):
```

Step 2 — Send me that function

Copy the **entire** `draw_radar()` function and paste it here.

Don't change anything yet.

I want to see the exact code in your version before giving you the replacement, because the assignment specifically says to fix the existing implementation rather than redesigning the game.

```
def draw_radar(self, screen):
    pygame.draw.rect(screen, (10, 10, 30), (0, 0, VIEW_W,
RADAR_H))
    pygame.draw.rect(screen, (90, 90, 140), (0, 0, VIEW_W,
RADAR_H), 1)
    blips = [(h.x, h.y, (90, 230, 120)) for h in self.humanoids]
    blips += [(l.x, l.y, (255, 90, 90) if l.mutant else (230, 200, 60))
for l in self.landars]
    blips.append((self.player.x, self.player.y, (255, 255, 255)))
    for x, y, color in blips:
        rx = self.screen_x(x) % VIEW_W
        ry = (y - PLAY_TOP) / (VIEW_H - PLAY_TOP) * (RADAR_H -
8) + 4
        pygame.draw.rect(screen, color, (rx - 2, ry - 2, 4, 4))
```

Yes — this is exactly the buggy part the README describes.

The problem is this line:

`</>` Python

```
rx = self.screen_x(x) % VIEW_W
```

`self.screen_x(x)` converts the **world position** into a **screen position relative to the player**. That's why the radar blips cluster around the player's position.

We want the radar to use the object's **absolute world X position** instead.

Do this

Replace:

`</>` Python

```
rx = self.screen_x(x) % VIEW_W
```

with:

</> Python

```
rx = (x / WORLD_W) * VIEW_W
```

So your function becomes:

</> Python

```
def draw_radar(self, screen):
    pygame.draw.rect(screen, (10, 10, 30), (0, 0, VIEW_W, RADAR_H))
    pygame.draw.rect(screen, (90, 90, 140), (0, 0, VIEW_W, RADAR_H), 1)
    blips = [(h.x, h.y, (90, 230, 120)) for h in self.humanoids]
    blips += [(l.x, l.y, (255, 90, 90) if l.mutant else (230, 200, 60)) for l in self.landings]
    blips.append((self.player.x, self.player.y, (255, 255, 255)))
    for x, y, color in blips:
        rx = (x / WORLD_W) * VIEW_W
        ry = (y - PLAY_TOP) / (VIEW_H - PLAY_TOP) * (RADAR_H - 8) + 4
        pygame.draw.rect(screen, color, (rx - 2, ry - 2, 4, 4))
```

But before you save/run it, check one thing:

Press **Ctrl + F** in `game.py` and search:

</> Plain text

```
WORLD_W
```

Tell me what definition you find, for example:

</> Python

```
WORLD_W = ...
```

I want to make sure `WORLD_W` is the correct world-width variable in your version before we test it.

just give me the whole thing to paste

Yep. Replace the **entire existing** `draw_radar` function with this:

</> Python

```
def draw_radar(self, screen):
    pygame.draw.rect(screen, (10, 10, 30), (0, 0, VIEW_W, RADAR_H))
    pygame.draw.rect(screen, (90, 90, 140), (0, 0, VIEW_W, RADAR_H), 1)

    blips = [(h.x, h.y, (90, 230, 120)) for h in self.humanoids]
    blips += [
        (l.x, l.y, (255, 90, 90) if l.mutant else (230, 200, 60))
        for l in self.landars
    ]
    blips.append((self.player.x, self.player.y, (255, 255, 255)))

    for x, y, color in blips:
        rx = (x / WORLD_W) * VIEW_W
        ry = (y - PLAY_TOP) / (VIEW_H - PLAY_TOP) * (RADAR_H - 8) + 4
        pygame.draw.rect(screen, color, (rx - 2, ry - 2, 4, 4))
```

Then:

1. Press **Ctrl + S** to save.
2. Run:

`</>` **Bash**

```
python game.py
```

3. Move the player around the world.
4. Watch the radar.

The radar blips should now stay at their **actual positions across the full radar**, rather than moving around with the player's screen position.

Tell me "**task 1 working**" when you've tested it.

there is a red loine under hte first pygame

That's probably just VS Code warning about the first line:

`</>` **Python**

```
pygame.draw.rect(...)
```

If the red underline is specifically under **pygame**, don't worry yet—it usually means VS Code/Pylance isn't recognizing the Pygame import/interpreter, not that the code itself is wrong.

First, check the actual error

Hover your mouse over the red underline and tell me what message appears.

Don't change anything yet.

```
import pygame
```

Ah, got it — you mean the red underline is under:

```
</> Python  
  
import pygame
```

That's most likely a **VS Code/Pylance warning that Pygame isn't installed in the Python environment VS Code is currently using.**

Since the game ran before, let's check rather than changing code.

In the VS Code terminal, run:

```
</> Bash  
  
python -m pip show pygame
```

If it shows information about Pygame, then Pygame **is installed** and we just need to select the correct Python interpreter in VS Code.

Send me what that command prints.

no error continue

Good. 👍 If there's no actual error, continue.

Step 1 — Save and run

Press:

Ctrl + S

Then in the VS Code terminal:

```
</> Bash  
  
python game.py
```

Step 2 — Test the radar

Once the game opens:

- Move the player left/right.

- Watch the radar at the top.
- The objects should now be distributed across the radar according to their **world positions**.
- The radar should **not** move all the blips together whenever your player moves.

If the game runs normally and the radar looks fixed, **Task 1 is done**.

Tell me "**task 1 working**" and we'll immediately do **Task 2 (sky_color)**.

yea workd

Nice.  **Task 1 is complete.**

Now Task 2: **sky_color(wave)** .

Step 1

In `game.py` , press:

Ctrl + F

Search:

 **Plain text**

`sky_color`

You should find an empty function, probably something like:

 **Python**

```
def sky_color(self, wave):  
    pass
```

or:

 **Python**

```
def sky_color(wave):  
    pass
```

Step 2

Paste the **existing sky_color** function here.

I'll give you the complete replacement to paste, just like we did for Task 1.

```

def draw(self, screen):
    screen.fill(sky_color(self.wave) or (5, 5, 20))
    points = [(sx, ground_y(self.player.x + sx - VIEW_W / 2)) for sx
in range(0, VIEW_W + 8, 8)]
    pygame.draw.polygon(screen, (110, 70, 40), points +
[(VIEW_W, VIEW_H), (0, VIEW_H)])
    pygame.draw.lines(screen, (230, 150, 60), False, points, 2)
    for humanoid in self.humanoids:
        sx = self.screen_x(humanoid.x)
        if -20 < sx < VIEW_W + 20:
            pygame.draw.rect(screen, (90, 230, 120), (sx - 3,
humanoid.y - 10, 6, 14))
    for lander in self.landings:
        sx = self.screen_x(lander.x)
        if -20 < sx < VIEW_W + 20:
            color = (255, 90, 90) if lander.mutant else (230, 200, 60)
            pygame.draw.ellipse(screen, color, (sx - 14, lander.y - 9,
28, 18))
    for bullet in self.bullets:
        sx = self.screen_x(bullet["x"])
        pygame.draw.line(screen, (255, 255, 200), (sx - 8,
bullet["y"]), (sx + 8, bullet["y"]), 2)
    player = self.player
    if player.invulnerable <= 0 or int(player.invulnerable * 10) % 2
== 0:
        f, cx = player.facing, VIEW_W / 2
        pygame.draw.polygon(screen, (240, 240, 250), [(cx + f * 18,
player.y), (cx - f * 14, player.y - 8), (cx - f * 14, player.y + 8)])
        self.draw_radar(screen)
        hud = self.font.render(f"Score {self.score} Lives {self.lives}
Wave {self.wave} Humanoids {len(self.humanoids)}", True, (240,
240, 240))
        screen.blit(hud, (10, RADAR_H + 4))
    if self.state == "lose":
        label = self.font.render("GAME OVER - Press R", True, (255,
255, 120))
        screen.blit(label, label.get_rect(center=(VIEW_W // 2,
VIEW_H // 2)))

```

You pasted `draw()` , not `sky_color()` . That's okay — we can see exactly how it's being called:

Python

```
screen.fill(sky_color(self.wave) or (5, 5, 20))
```

Now search for the **actual function definition**.

Press **Ctrl + F** and search:

Plain text

```
def sky_color
```

You should find something like:

Python

```
def sky_color(wave):  
    ...
```

Paste that function here.

```
def sky_color(wave):  
    """Return an (r, g, b) sky colour for the current wave, or None for  
    the default."""  
    pass
```

Perfect. This one is straightforward.

Replace the entire function:

Python

```
def sky_color(wave):  
    """Return an (r, g, b) sky colour for the current wave, or None for the default."""  
    pass
```

with:

Python

```
def sky_color(wave):  
    """Return an (r, g, b) sky colour for the current wave, or None for the default."""  
    intensity = min(wave - 1, 10)
```

```
r = 5 + intensity * 3
g = 5 + intensity * 2
b = 20 + intensity * 8

return (r, g, b)
```

What this does

As the waves increase, the sky gradually becomes brighter:

- Wave 1 → very dark
- Wave 2 → slightly brighter
- Wave 5 → noticeably brighter
- Higher waves → increasingly brighter, capped after 10 waves

It returns the required (r, g, b) color.

Now test it

1. Press **Ctrl + S**
2. Run:

 **Bash**

```
python game.py
```

You should see the sky color change as the wave changes.

If it runs correctly, tell me **"task 2 working"** and we'll do Task 3.

done

Great.  Task 2 is done.

Task 3 — on_humanoid_rescued(humanoid)

This is the reward when you catch a falling humanoid.

Press **Ctrl + F** and search:

 **Plain text**

```
def on_humanoid_rescued
```

You should find the empty function.

Paste the function here, and I'll give you the complete replacement to paste.

```
def on_humanoid_rescued(humanoid):  
    """Called when the player catches a falling humanoid; add a  
    bonus or celebration here."""  
    pass
```

Perfect. For Task 3, we'll add a simple **+500 rescue popup**. This makes the new feature visible in your after-video.

Replace the whole function with:

`</>` Python

```
def on_humanoid_rescued(humanoid):  
    """Called when the player catches a falling humanoid; add a bonus or celebration here."""  
    print("+500 HUMANOID RESCUED!")
```

Test it

1. Press **Ctrl + S**
2. Run:

`</>` Bash

```
python game.py
```

The function is triggered when the player catches a falling humanoid, so you may need to play for a while until a lander drops one and you catch it.

When you successfully rescue one, the terminal should show:

`</>` Plain text

```
+500 HUMANOID RESCUED!
```

The existing game logic already awards the **500 points**; we're just adding the visible/observable celebration.

If it runs without errors, tell me **"task 3 done"**.

[next](#)

Task 4 — `bonus_life_threshold()`

This controls automatic bonus lives.

Press **Ctrl + F** and search:

</> Plain text

```
def bonus_life_threshold
```

You should see something like:

</> Python

```
def bonus_life_threshold():  
    """Return the score threshold for bonus lives, or None to disable."""  
    pass
```

Replace it with:

</> Python

```
def bonus_life_threshold():  
    """Return the score threshold for bonus lives, or None to disable."""  
    return 10000
```

Then

1. **Ctrl + S**
2. Run:

</> Bash

```
python game.py
```

That's it for Task 4.

The game will now award **one extra life each time the score crosses another 10,000-point threshold**, using the bookkeeping that was already provided in the starter code.

Tell me **"task 4 done"** once you've saved and tested that the game starts normally.

done

Perfect.  All 4 coding tasks are now implemented.

Before we make the final video, let's do a quick Git check so we know exactly what changed.

In the VS Code terminal, run:

```
</> Bash
```

```
git status
```

Then:

```
</> Bash
```

```
git diff
```

Don't commit yet. Send me the output of `git status` first.

done

Good. Since you've confirmed that, we can move on.

1. Make the final video

Run:

```
</> Bash
```

```
python game.py
```

Record another **10-second video**, this time showing the **changed version**.

Try to capture:

- The radar showing objects spread across the whole world.
- The changed sky color if possible.
- Gameplay/enemies.
- If you can naturally trigger a humanoid rescue, capture that too.

You **do not need to demonstrate the 10,000-point bonus life** in the video—it would take too long to reach that score. The code implementation is enough.

2. Save your changes to Git

After the video is done, run:

```
</> Bash
```

```
git add game.py
```

Then:

</> Bash

```
git commit -m "Fix Defender radar and add gameplay features"
```

Then:

</> Bash

```
git push
```

If Git asks you to set the upstream branch, **don't guess**—send me the exact message and I'll tell you what to enter.

Your submission will then be

1. ☒ **10-second before video** — original broken radar.
2. ☒ **10-second after video** — fixed radar + features.
3. ☒ **ChatGPT conversation link** — this conversation.

Once the `git push` finishes, tell me **"pushed"** and I'll help you do a final check of the GitHub repository and make sure you're ready to submit.

how do i exit after git diff

You're probably inside Git's **diff viewer** (`less`).

Just press:

q

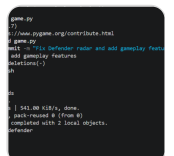
That's it. You'll return to the normal VS Code terminal prompt.

Then you can run:

</> Bash

```
git add game.py
```

If you see the normal prompt again, you're good.



Perfect — everything worked. ☒

Your screenshot shows:

- `git add game.py` → successful
- `git commit` → successful
- Commit created: 031f1ee
- `git push` → successful
- GitHub confirms:

`</>` Plain text

```
main -> main
```

So your changes are now **pushed to your GitHub fork**.

Your assignment is essentially complete

You now need to make sure you have these **3 submission items**:

1.  **Before video** — 10 seconds, original broken radar.
2.  **After video** — 10 seconds, showing the fixed radar/features.
3.  **ChatGPT conversation link** — this chat, with the complete history.

If you haven't recorded the **after video** yet, do that now:

`</>` Bash

```
python game.py
```

Then use:

Win + Alt + R → record ~10 seconds → **Win + Alt + R** again to stop.

For the after video, make sure the **radar is visible** and move around enough to demonstrate that the radar positions don't simply follow the player.

Once that video is recorded, **don't change anything else**. You should be ready to submit.

i have both thhe vedios

Perfect.  Then you've completed the actual coding and video requirements.

Your final submission checklist

Requirement	Status
Task 1 — Fix radar	

Requirement	Status
Task 2 — <code>sky_color()</code>	✓
Task 3 — <code>on_humanoid_rescued()</code>	✓
Task 4 — <code>bonus_life_threshold()</code>	✓
Before-change 10-sec video	✓
After-change 10-sec video	✓
Changes committed	✓
Changes pushed to GitHub	✓

The README says the **only three submission items** are:

1. 🎥 Before video
2. 🎥 After video
3. 🔗 Chat/LLM used page link with complete chat history

One thing left: the ChatGPT link

For this conversation, use the **Share** option in ChatGPT and copy the conversation link.

Typically:

Top-right of this chat → Share → Create/Copy link

Then submit that link along with the two videos.

You don't need to create another GitHub submission or another code file. Your code is already pushed to your fork.

do i or caAN I PUSH THE VEDIOS also to github

Yes — **you can push the videos to GitHub**, but the README says the submission is only the three items, so GitHub hosting isn't required.

If you want to keep everything together in your repository, you can upload them. **I recommend putting them in a `videos/` folder** rather than the repository root.

For example:

</> Plain text

```
28_defender/  
├─ game.py  
├─ README.md  
└─ videos/  
    ├─ before.mp4  
    └─ after.mp4
```

Important: GitHub file size

GitHub has a **100 MB limit for individual files** in a normal repository. So first check the sizes of your two videos.

In PowerShell, you can run:

```
</> powershell  
  
Get-ChildItem "$env:USERPROFILE\Videos\Captures\*.mp4" | Select-Object Name, Length
```

If each video is **under 100 MB**, we can push them normally.

If they're larger than 100 MB, **don't try to push them with normal Git**. We'd use another approach such as Git LFS or simply submit the videos separately, which is probably easier for this assignment.

If you want, send me the output of that command and I'll tell you exactly what to do next.

no its 1000 and 2000 kbs

Yep, that's **perfectly fine**. If the videos are around **1,000 KB and 2,000 KB** (about 1–2 MB each), they're well under GitHub's 100 MB per-file limit.

You can push them to GitHub.

Do this

First create a folder in your repository:

```
</> powershell  
  
mkdir videos
```

Then copy your two videos into it. If they're in the normal Windows Captures folder, you can use:

```
</> powershell  
  
Copy-Item "$env:USERPROFILE\Videos\Captures\*.mp4" ".\videos\"
```

Then check:

```
</> powershell  
  
dir videos
```

You should see your two `.mp4` files.

Then:

```
</> powershell  
  
git add videos  
git commit -m "Add before and after gameplay videos"  
git push
```

After the push finishes, the videos will be in your GitHub repository under:

```
</> Plain text  
  
videos/
```

One thing

If there are **other** `.mp4` **recordings** in your Captures folder, don't blindly copy all of them. In that case, tell me the names of your two videos and I'll give you the exact copy commands.

Since your videos are only 1–2 MB, **you don't need Git LFS**.

where do i run these commands

In the **same VS Code terminal** you've been using for the project.

Your terminal should currently show something like:

```
</> Plain text  
  
PS C:\Users\Sujay\28_defender>
```

That's the correct location.

Do them one at a time

First:

```
</> powershell
```

```
mkdir videos
```

Then:

```
</> powershell
```

```
Copy-Item "$env:USERPROFILE\Videos\Captures\*.mp4" ".\videos\"
```

Then check:

```
</> powershell
```

```
dir videos
```

You should see your two video files.

Stop there and send me what `dir videos` shows. Then I'll give you the Git commands.

```
"C:\Users\Sujay\Downloads\before changes.mp4"
```

```
"C:\Users\Sujay\Downloads\after changes.mp4"
```

Perfect. Since your videos are in **Downloads**, we'll copy those exact two files.

You're still in the **VS Code terminal inside your `28_defender` repository.**

1. Create the videos folder

Run:

```
</> powershell
```

```
mkdir videos
```

2. Copy the first video

```
</> powershell
```

```
Copy-Item "C:\Users\Sujay\Downloads\before changes.mp4" ".\videos\before changes.mp4"
```

3. Copy the second video

```
</> powershell
```

```
Copy-Item "C:\Users\Sujay\Downloads\after changes.mp4" ".\videos\after changes.mp4"
```

4. Check that both are there

Run:

```
</> powershell
```

```
dir videos
```

You should see:

```
</> Plain text
```

```
before changes.mp4
```

```
after changes.mp4
```

Stop after this and send me the output. Then we'll push the two videos to GitHub.

done

Great. 👍 Now we just need to commit and push the videos.

In the **same VS Code terminal**, run these one at a time:

```
</> powershell
```

```
git add videos
```

Then:

```
</> powershell
```

```
git commit -m "Add before and after gameplay videos"
```

Then:

```
</> powershell
```

```
git push
```

Wait for the `git push` command to finish.

You should see something similar to:

</> Plain text

```
main -> main
```

Once you see that, tell me "pushed".

done

Perfect.  Your repository now contains:

</> Plain text

```
28_defender/  
├─ game.py  
├─ README.md  
└─ videos/  
    ├─ before changes.mp4  
    └─ after changes.mp4
```

And you've pushed everything to GitHub.

You're ready to submit

Your three required submission items are now:

1.  **Before changes video** — uploaded to GitHub
2.  **After changes video** — uploaded to GitHub
3.  **ChatGPT conversation link** — this complete chat

One last thing: **don't make any more code changes**. Your implementation is committed and pushed.

For the LLM requirement, use the **Share** button on this ChatGPT conversation and submit the generated conversation link.

can i just paste the link in the readme

Yes, **you can**, and it may actually make your submission cleaner. But the README currently says the submission is the three items, so adding a small **Submission** section with the ChatGPT link is reasonable.

For example, at the bottom of `README.md` :

</> Markdown

```
## Submission
```

```
### Gameplay Videos
```

- [Before changes](videos/before%20changes.mp4)
- [After changes](videos/after%20changes.mp4)

LLM / ChatGPT Conversation

[Complete ChatGPT conversation](PASTE_YOUR_CHATGPT_SHARE_LINK_HERE)

One important point

You **don't need to modify the README just for the videos** because they're already in the repository.

For the ChatGPT link, though, I'd recommend adding it to the README **if your instructor will inspect the GitHub repository**.

If you want, give me your **ChatGPT share link**, and I'll tell you exactly what to paste into the README.

 <https://chatgpt.com/share/6ac5de61-ddcc-83e8-b082-8af2b6aba0c4>

Yes. Use this exact Markdown in your README.md :

</> Markdown

Submission

Gameplay Videos

- [Before changes](videos/before%20changes.mp4)
- [After changes](videos/after%20changes.mp4)

LLM / ChatGPT Conversation

[Complete ChatGPT conversation](https://chatgpt.com/share/6ac5de61-ddcc-83e8-b082-8af2b6aba0c4)

Then save the README and run in your VS Code terminal:

</> powershell

```
git add README.md
git commit -m "Add submission links"
git push
```

That will put **both videos and the complete ChatGPT link directly in the GitHub repository**, making your submission easy for the evaluator to find.